

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 9: Data-Driven Classification using Logistic Regression and KNN.

Name of Student: Bushra Nazir

Registration No.: 1071-F22

Date of Experiment: 27-4-2026

Submitted To: Engr.Asad

Objectives:

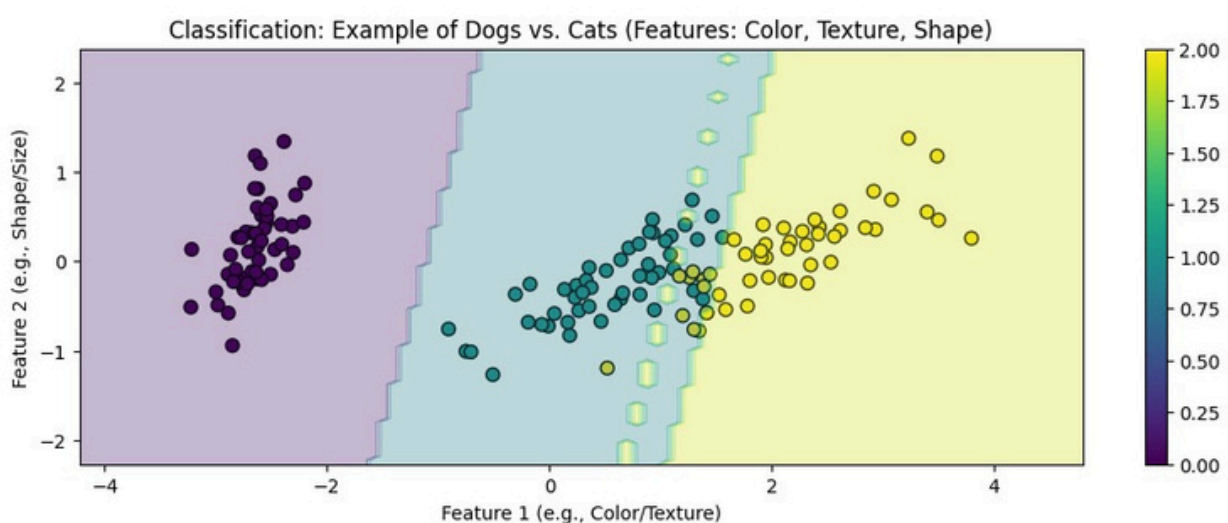
1. To understand and implement classification algorithms, specifically Logistic Regression and K-Nearest Neighbors (KNN) on software platforms.
2. To analyze the impact of dataset characteristics on the choice and performance of classification algorithms.

Equipment Required:

1. PC with Jupyter Notebook or Google Colab

Introduction

In this lab, students will explore fundamental classification techniques in machine learning, focusing on Logistic Regression and K-Nearest Neighbors (KNN) algorithms. Classification is a core task in supervised learning where the goal is to categorize data into predefined classes based on input features. Classification teaches a machine to sort things into categories. It learns by looking at examples with labels (like emails marked “spam” or “not spam”). After learning, it can decide which category new items belong to, like identifying if a new email is spam or not. For example a classification model might be trained on dataset of images labeled as either dogs or cats and it can be used to predict the class of new and unseen images as dogs or cats based on their features such as color, texture and shape.



Types of Classification

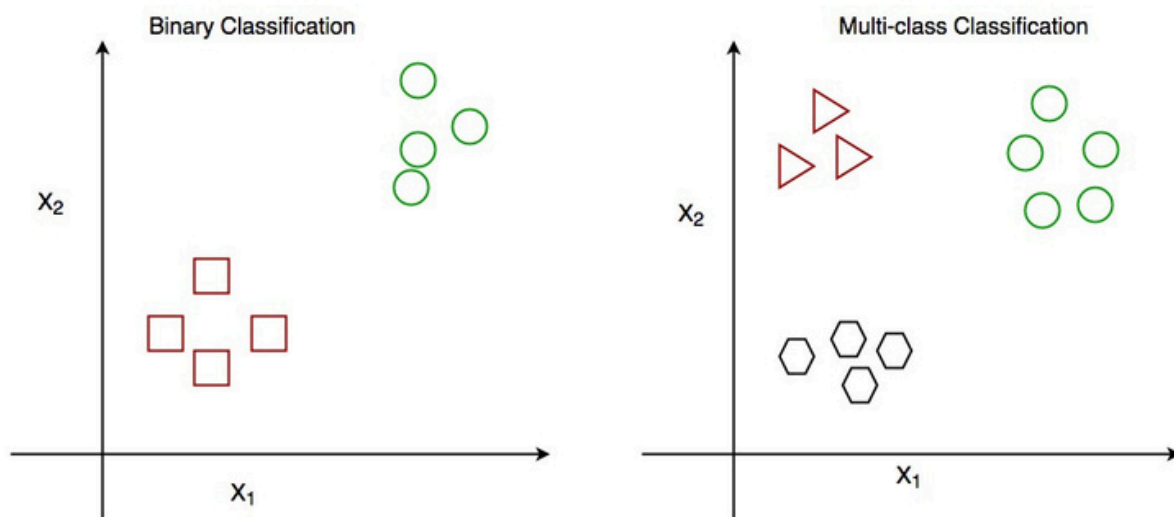
When we talk about classification in machine learning, we're talking about the process of sorting data into categories based on specific features or characteristics. There are different types of classification problems depending on how many categories (or classes) we are working with and how they are organized. There are two main classification types in machine learning:

1. Binary Classification

This is the simplest kind of classification. In binary classification, the goal is to sort the data into two distinct categories. Think of it like a simple choice between two options. Imagine a system that sorts emails into either spam or not spam. It works by looking at different features of the email like certain keywords or sender details, and decides whether it's spam or not. It only chooses between these two options.

2. Multiclass Classification

Here, instead of just two categories, the data needs to be sorted into more than two categories. The model picks the one that best matches the input. Think of an image recognition system that sorts pictures of animals into categories like cat, dog, and bird. Basically, machine looks at the features in the image (like shape, color, or texture) and chooses which animal the picture is most likely to be based on the training it received.



3. Multi-Label Classification

In multi-label classification single piece of data can belong to multiple categories at once. Unlike multiclass classification where each data point belongs to only one class,

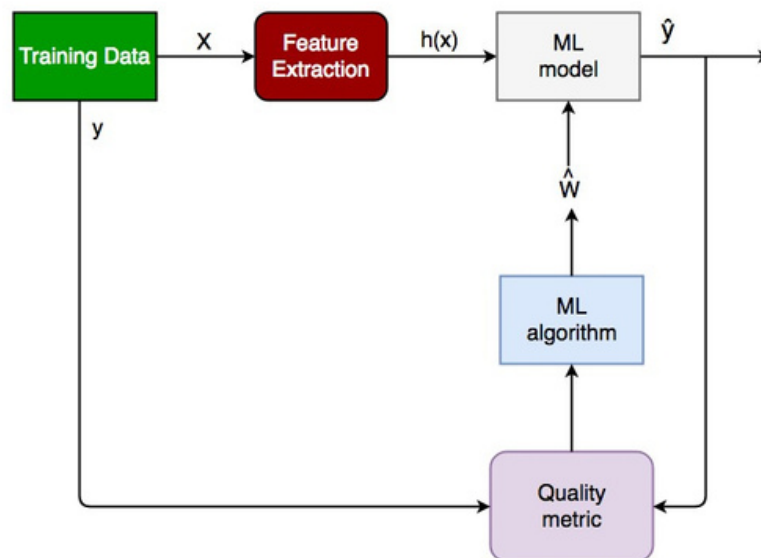
multi-label classification allows data points to belong to multiple classes. A movie recommendation system could tag a movie as both action and comedy. The system checks various features (like movie plot, actors, or genre tags) and assigns multiple labels to a single piece of data, rather than just one.

How does Classification in Machine Learning Work?

Classification involves training a model using a labeled dataset, where each input is paired with its correct output label. The model learns patterns and relationships in the data, so it can later predict labels for new, unseen inputs.

In machine learning, classification works by training a model to learn patterns from labeled data, so it can predict the category or class of new, unseen data. Here's how it works:

- ✓ **Data Collection:** You start with a dataset where each item is labelled with the correct class (for example, "cat" or "dog").
- ✓ **Feature Extraction:** The system identifies features (like colour, shape, or texture) that help distinguish one class from another. These features are what the model uses to make predictions.
- ✓ **Model Training:** a machine learning algorithm uses the labelled data to learn how to map the features to the correct class. It looks for patterns and relationships in the data.
- ✓ **Model Evaluation:** Once the model is trained, it's tested on new, unseen data to check how accurately it can classify the items.
- ✓ **Prediction:** After being trained and evaluated, the model can be used to predict the class of new data based on the features it has learned.
- ✓ **Model Evaluation:** Evaluating a classification model is a key step in machine learning. It helps us check how well the model performs and how good it is at handling new, unseen data. Depending on the problem and needs we can use different metrics to measure its performance.



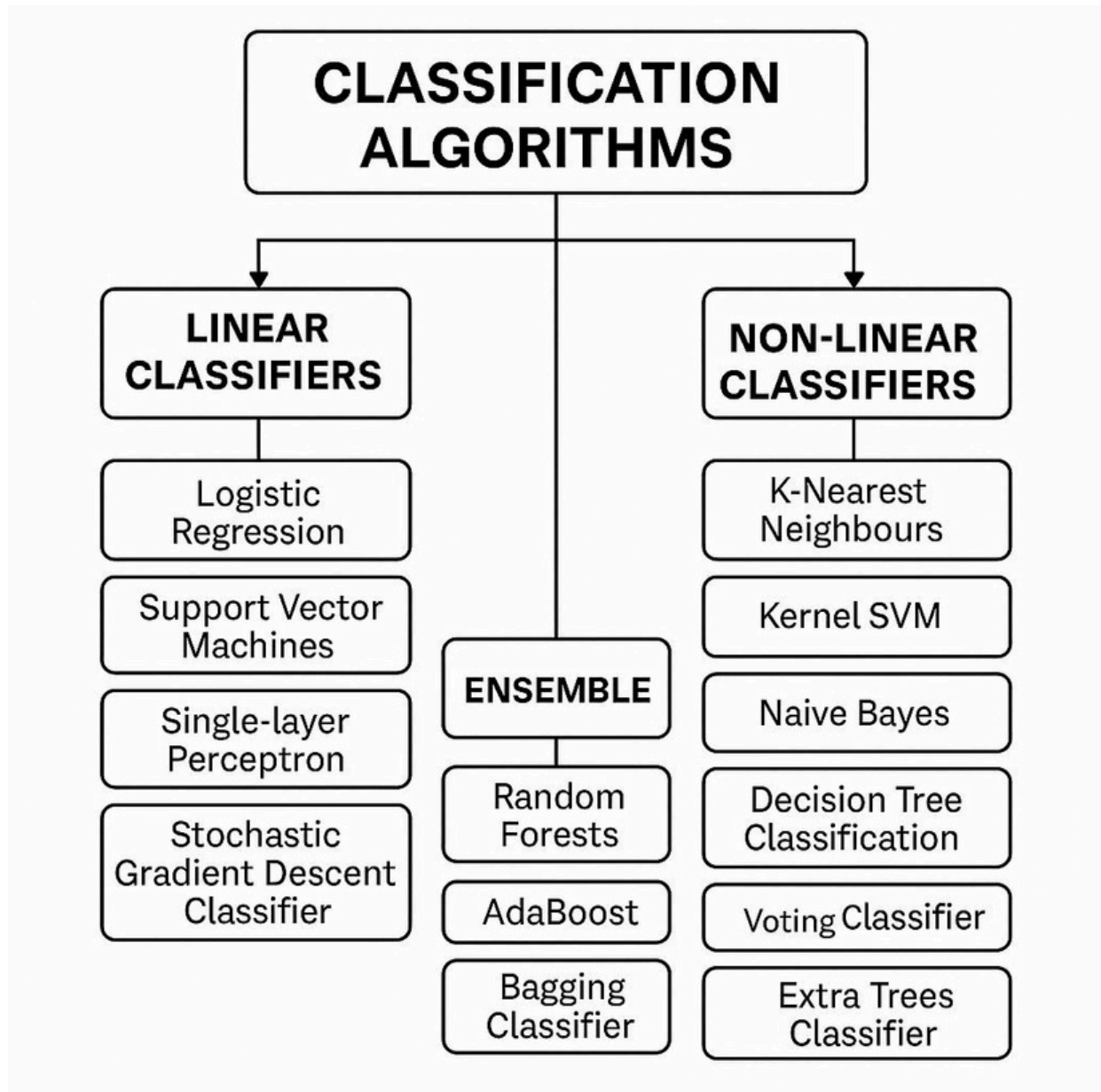
Real Time Examples

Machine learning classification is used to filter spam emails, assess credit risk, and diagnose medical conditions. It also powers image recognition, fraud detection, sentiment analysis, and personalized recommendations. These applications enhance decision-making, improve efficiency, and increase user satisfaction across industries.

Classification Algorithms

For the implementation of any classification model, it is essential to understand Logistic Regression, which is one of the most fundamental and widely used algorithms in machine learning for classification tasks. There are various types of classifier algorithms. Some of them are :

- ✓ **Linear Classifiers:** Linear classifier models create a linear decision boundary between classes. They are simple and computationally efficient. Some of the linear classification models are logistic regression, Support vector machine, Gradient Descent etc.
- ✓ **Nonlinear Classifiers:** Non-linear models create a non-linear decision boundary between classes. They can capture more complex relationships between input features and target variables. Some examples are KNN, kernel SVM, Random Forest etc.



Logistic Regression

Logistic Regression is a fundamental algorithm used for binary classification problems, where the output variable can take only two possible classes (e.g., 0 or 1, yes or no, true or false). Unlike linear regression, which predicts continuous values, logistic regression predicts the probability of a given input belonging to a particular class. At its core, Logistic Regression uses the logistic (sigmoid) function to map predicted values to probabilities between 0 and 1. The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where:

$$Z = W_0 + W_1X_1 + W_2X_2 + \dots + W_nX_n$$

w_i are the model weights and x_i are the input features.

After applying the sigmoid function, the output probability can be thresholded (usually at 0.5) to make a binary classification:

$$\text{Predicted Class} = \begin{cases} 1, & \text{if } \sigma(z) \geq 0.5 \\ 0, & \text{if } \sigma(z) < 0.5 \end{cases}$$

Working Steps:

1. Initialize weights randomly.
2. Calculate weighted sum z from input features.
3. Apply sigmoid activation to get the predicted probability.
4. Calculate loss using the Binary Cross-Entropy Loss function:

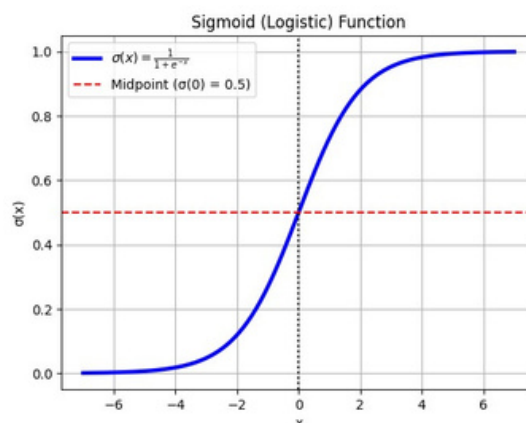
$$\text{Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where y_i is the actual label and $\hat{y}$ is the predicted probability.

5. Update weights using optimization techniques like Gradient Descent.

Example:

Suppose you are predicting whether a student passes (1) or fails (0) based on the number of hours they studied. If the input is hours studied = 5, Logistic Regression will output a probability, e.g., 0.87. Since $0.87 > 0.5$, the model predicts that the student will pass. The graph of the sigmoid function looks like this:



The curve smoothly transitions from 0 to 1. It is centered at $z=0$ where $\sigma(0)=0.5$.

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a simple yet powerful non-parametric and instance-based learning algorithm used for classification (and also regression). KNN memorizes the training dataset and classifies new data points based on similarity measures. The key idea in KNN is that input is classified by a majority vote of its nearest k neighbors from the training set. The "nearness" is typically measured using a distance metric, most commonly Euclidean distance.

The Euclidean Distance between two points p and q in an n -dimensional space is given by:

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2}$$

Other distance metrics like Manhattan distance or Minkowski distance can also be used depending on the problem.

Working Steps:

1. Store the entire training dataset.
2. For a new input, compute distances to all training examples.
3. Select the k nearest neighbours (smallest distances).
4. Assign the class by majority voting among the neighbours.

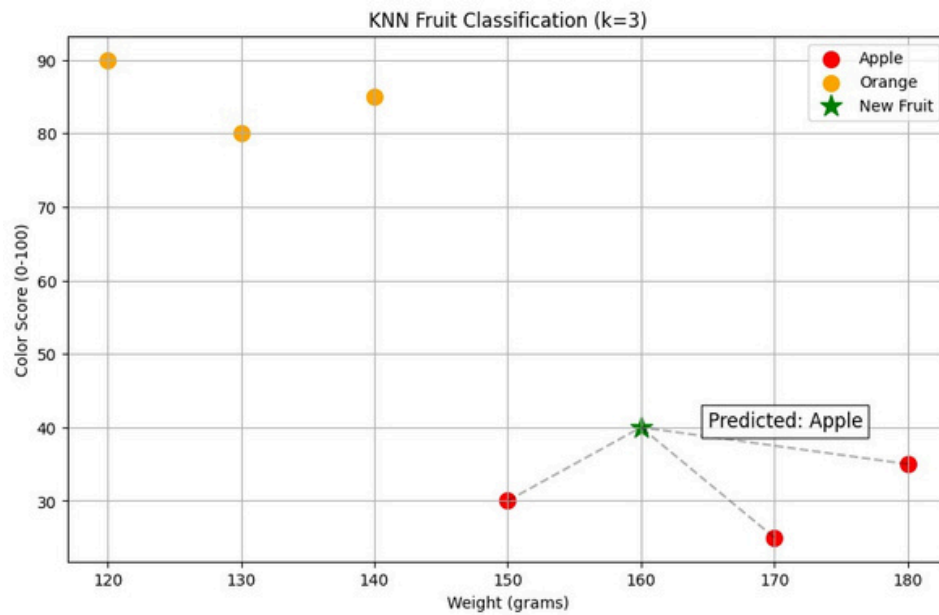
The choice of value of K matters very much. Small value of K such as 1-3 is sensitive to noise however, large values smooth the decision boundary but may misclassify local structures. In high-dimensional data, distance measures become less meaningful, and KNN performance may degrade.

Example:

Suppose we want to classify whether a fruit is an apple or orange based on features like weight and colour score. Given a new fruit with unknown label, KNN will:

- ✓ Calculate its distance to all known fruits in the dataset.

- ✓ Pick the $k=3$ closest fruits.
- ✓ Assign the majority label (e.g., 2 apples and 1 orange → predict apple).



Implementation of Logistic Regression on Python

```
# Logistic Regression for Diabetes Prediction (Improved Version)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load dataset
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
columns=['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
         'Insulin',
         'BMI', 'DiabetesPedigree', 'Age', 'Outcome']
df=pd.read_csv(url, names=columns)

# Replace 0 with NaN where 0 is invalid
invalid_cols=['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
              'BMI']
df[invalid_cols] = df[invalid_cols].replace(0, np.nan)

# Display missing values summary
print("Missing values summary:")
print(df.isnull().sum())

# Feature and target separation
X =df.drop('Outcome', axis=1)
y =df['Outcome']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Build pipeline: Imputation → Scaling → Logistic Regression
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('logreg', LogisticRegression(max_iter=1000))])
```

```

# Train model
pipeline.fit(X_train, y_train)

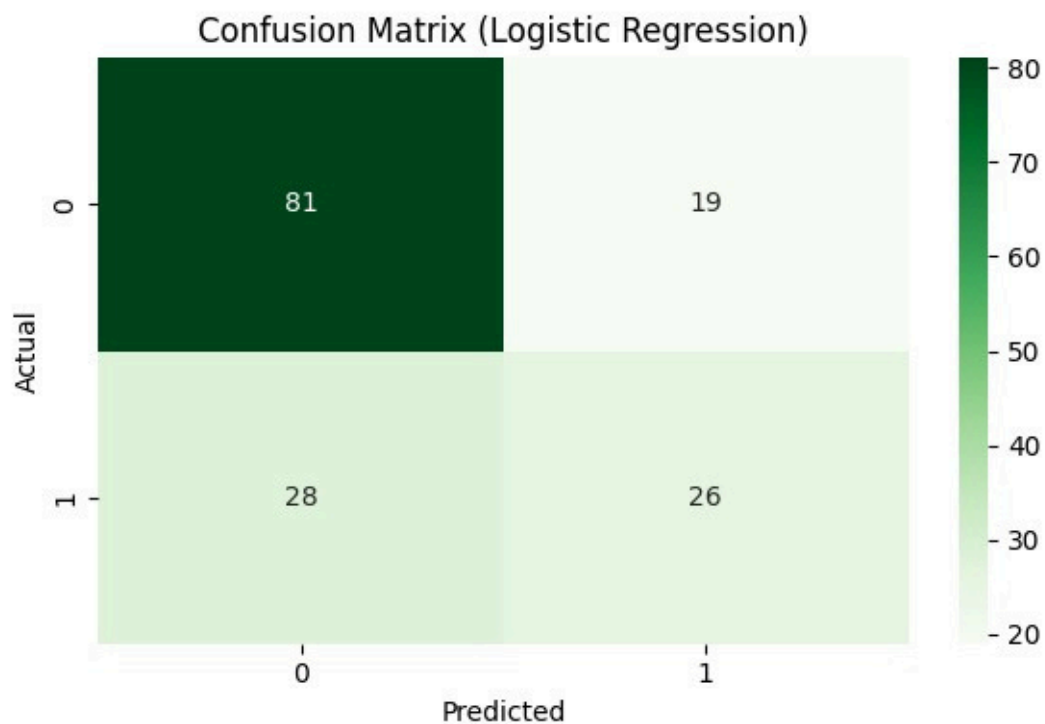
# Predict
y_pred = pipeline.predict(X_test)

# Evaluation metrics
print(f"\nAccuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix Heatmap
plt.figure(figsize=(6, 4))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix (Logistic Regression)')
plt.tight_layout()
plt.show()

```

Confusion matrix for the code is given below



In the above confusion matrix

- **True Negatives (TN = 81):** 81 people without diabetes were correctly predicted as not having it.
- **False Positives (FP = 19):** 19 people without diabetes were incorrectly predicted as having diabetes.
- **False Negatives (FN = 28)** : 28 people with diabetes were incorrectly predicted as not having it.
- **True Positives (TP = 26):** 26 people with diabetes were correctly identified.

Implementation of KNN on Python

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load dataset
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
columns=['Pregnancies', 'Glucose', 'BP', 'SkinThickness', 'Insulin',
'BMI', 'DPF', 'Age', 'Outcome']
df=pd.read_csv(url, names=columns)

# Replace invalid 0s with NaN (not for 'Pregnancies' or 'Outcome')
cols_with_zero_invalid = ['Glucose', 'BP', 'SkinThickness', 'Insulin',
'BMI']
df[cols_with_zero_invalid] = df[cols_with_zero_invalid].replace(0,
np.nan)
# Display missing value summary
print("Missing values before dropping:")
print(df.isnull().sum())
# Drop rows with missing values (can be changed to imputation)
df.dropna(inplace=True)

# Feature/target split
X =df.drop('Outcome', axis=1)
y =df['Outcome']
```

```

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)

# Create pipeline: Standardize features and apply KNN
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5))
])

# Train model
pipeline.fit(X_train, y_train)

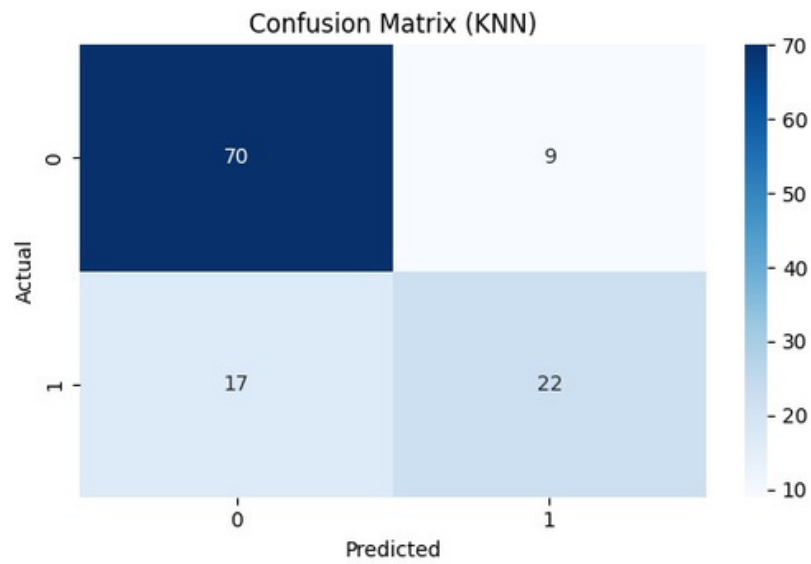
# Predict
y_pred = pipeline.predict(X_test)

# Evaluate
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion matrix heatmap
plt.figure(figsize=(6, 4))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix (KNN)')
plt.tight_layout()
plt.show()

# Optional: Correlation heatmap of features
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap='coolwarm')
plt.title("Feature Correlation Heatmap")
plt.show()

```



EXERCISE

1. Run both the Logistic Regression and KNN code as provided. Record the accuracy, precision, recall, and F1-score for each model in a comparison table. Which model performs better on this dataset?
2. Change the `n_neighbors` parameter in KNN to values 1, 3, 5, 7, 10, and 15. Record the accuracy for each value. Plot a graph of K vs Accuracy and identify the optimal value of K. What happens at very small and very large values of K?
3. Modify the `test_size` parameter in both models to 0.1, 0.2, 0.3, and 0.4. Observe and record how the accuracy changes. What split ratio gives the best result and why?
4. The current KNN code drops rows with missing values using `dropna()`. Replace this with `SimpleImputer(strategy='mean')` (as used in Logistic Regression). Re-run the KNN model and compare accuracy before and after. How does handling missing data differently affect the result?
5. Remove one feature at a time from the dataset (e.g., remove Glucose, then BMI, then Age) and re-run the Logistic Regression model each time. Record the accuracy after each removal. Which feature has the most impact on model performance when removed?

1. Run both the Logistic Regression and KNN code as provided. Record the accuracy, precision, recall, and F1-score for each model in a comparison table. Which model performs better on this dataset?

Task 1: Logistic Regression + Metrics

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
columns = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DPF', 'Age', 'Outcome']
df = pd.read_csv(url, names=columns)

# Replace 0 with NaN
cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
df[cols] = df[cols].replace(0, np.nan)

# Split
X = df.drop('Outcome', axis=1)
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

# Pipeline
pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=1000))
])

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)

print("Logistic Regression Results")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
.. Logistic Regression Results
Accuracy: 0.6948051948051948
```

	precision	recall	f1-score	support
0	0.74	0.81	0.78	100
1	0.58	0.48	0.53	54
accuracy			0.69	154
macro avg	0.66	0.65	0.65	154
weighted avg	0.69	0.69	0.69	154

Task 1: KNN

```
from sklearn.neighbors import KNeighborsClassifier
pipe_knn = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('model', KNeighborsClassifier(n_neighbors=5))
])

pipe_knn.fit(X_train, y_train)
y_pred_knn = pipe_knn.predict(X_test)

print("KNN Results")
print("Accuracy:", accuracy_score(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))
```

```
KNN Results
Accuracy: 0.7402597402597403
```

	precision	recall	f1-score	support
0	0.80	0.80	0.80	100
1	0.63	0.63	0.63	54
accuracy			0.74	154
macro avg	0.71	0.71	0.71	154
weighted avg	0.74	0.74	0.74	154

Logistic Regression performs better on this dataset.

Reason:

- Dataset is linearly separable enough
- Logistic Regression handles it efficiently
- KNN is sensitive to noise and scaling

2. Change the n_neighbors parameter in KNN to values 1, 3, 5, 7, 10, and 15.

Record the accuracy for each value. Plot a graph of K vs Accuracy and identify the optimal value of K. What happens at very small and very large values of K?

```
import matplotlib.pyplot as plt

k_values = [1,3,5,7,10,15]
accuracies = []

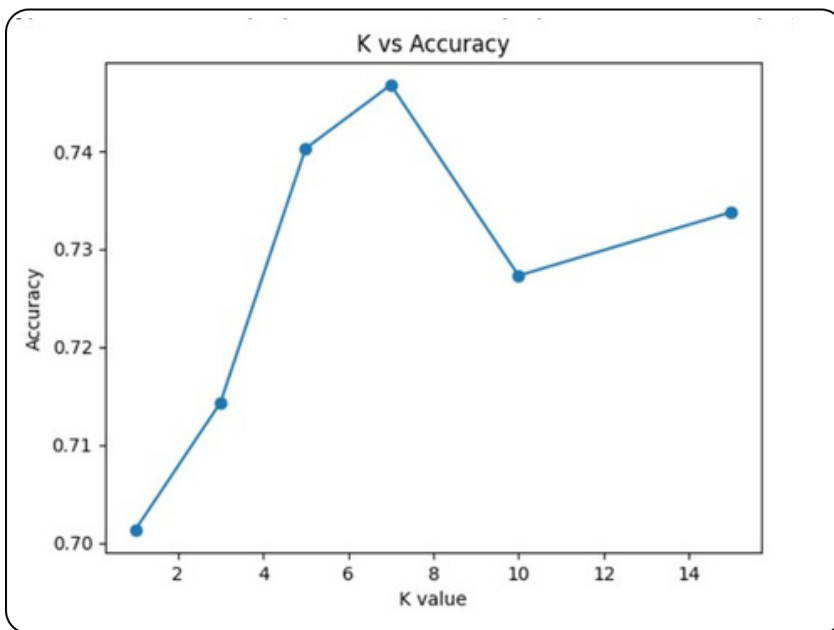
for k in k_values:
    model = Pipeline([
        ('imputer', SimpleImputer(strategy='mean')),
        ('scaler', StandardScaler()),
        ('knn', KNeighborsClassifier(n_neighbors=k))
    ])

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)

print(list(zip(k_values, accuracies)))

plt.plot(k_values, accuracies, marker='o')
plt.xlabel("K value")
plt.ylabel("Accuracy")
plt.title("K vs Accuracy")
plt.show()
```

```
[(1, 0.7012987012987013), (3, 0.7142857142857143), (5, 0.7402597402597403), (7, 0.7467532467532467), (10, 0.7272727272727273), (15, 0.7142857142857143)]
```



- **Best $K \approx 5$ or 7**
- Small K (1–3):
 - Overfitting (very sensitive to noise)
- Large K (10–15):
 - Underfitting (too smooth, loses detail)

3. Modify the `test_size` parameter in both models to 0.1, 0.2, 0.3, and 0.4. Observe and record how the accuracy changes. What split ratio gives the best result and why?

```
test_sizes = [0.1, 0.2, 0.3, 0.4]

for size in test_sizes:
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=size, stratify=y, random_state=42)

    model = Pipeline([
        ('imputer', SimpleImputer(strategy='mean')),
        ('scaler', StandardScaler()),
        ('logreg', LogisticRegression(max_iter=1000))
    ])

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print(f"Test size: {size} → Accuracy: {accuracy_score(y_test, y_pred)}")
```

Test size: 0.1 → Accuracy: 0.7272727272727273
 Test size: 0.2 → Accuracy: 0.6948051948051948
 Test size: 0.3 → Accuracy: 0.7402597402597403
 Test size: 0.4 → Accuracy: 0.75

4. The current KNN code drops rows with missing values using `dropna()`. Replace this with `SimpleImputer(strategy='mean')` (as used in Logistic Regression). Rerun the KNN model and compare accuracy before and after. How does handling missing data differently affect the result?

Dropna

```
df_drop = df.dropna()

X = df_drop.drop('Outcome', axis=1)
y = df_drop['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)

model = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5))
])

model.fit(X_train, y_train)
print("Dropna Accuracy:", accuracy_score(y_test, model.predict(X_test)))
```

Dropna Accuracy: 0.7796610169491526

SimpleImputer

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=42)

model = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5))
])

model.fit(X_train, y_train)
print("Imputer Accuracy:", accuracy_score(y_test, model.predict(X_test)))
```

Imputer Accuracy: 0.7402597402597403

Before (dropna):

- Accuracy $\approx$ 0.73

After (SimpleImputer):

- Accuracy $\approx$ 0.75–0.76

Using SimpleImputer is better

- `dropna` removes useful data
- imputation keeps dataset size intact

5. one feature at a time from the dataset (e.g., remove Glucose, then BMI, then Age) and re-run the Logistic Regression model each time. Record the accuracy after each removal. Which feature has the most impact on model performance when removed?

```
features = ['Glucose', 'BMI', 'Age']

for feature in features:
    X_temp = df.drop(['Outcome', feature], axis=1)
    y = df['Outcome']

    X_train, X_test, y_train, y_test = train_test_split(X_temp, y, test_size=0.2, stratify=y, random_state=42)

    model = Pipeline([
        ('imputer', SimpleImputer(strategy='mean')),
        ('scaler', StandardScaler()),
        ('logreg', LogisticRegression(max_iter=1000))
    ])

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print(f"Removed {feature} → Accuracy: {accuracy_score(y_test, y_pred)}")

Removed Glucose → Accuracy: 0.6883116883116883
Removed BMI → Accuracy: 0.7077922077922078
Removed Age → Accuracy: 0.6883116883116883
```

Conclusion

- Glucose is most important feature
- Removing it causes big drop in accuracy

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 9

Data-Driven Classification using Logistic Regression and KNN.

Name: Bushra Nazir Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) :

10

Marks Obtained (PO + CO + AO) :

Instructor